UNIDO AI Chatbot — System Flows

Complete technical documentation of every flow in the UNIDO Careers RAG Chatbot, from user query to AI response, data pipeline to admin operations.

V1.0 · APRIL 2026 · MOURITECH

High-Level Architecture

Component overview and technology stack

System Components

Frontend (React 19)
Vite 7 · Bootstrap 5 · Recharts · Socket.IO Client · React Hook Form

Backend (Express 5)
Node.js · Mongoose 9 · Puppeteer · Socket.IO · node-cron

Azure OpenAI
GPT-4o-mini (chat) · text-embedding-3-small (embeddings, 1536D)

Elasticsearch
Vector store · kNN cosine search · dense_vector index

MongoDB
Sessions · Chat Logs · Admin Users · Settings

Security Layer
Helmet · CORS · Rate Limits · JWT · bcrypt 12 · NoSQL Guard

Data Flow Overview

User (Browser) → React Frontend (ChatWidget.jsx) → Express API (chat.controller) → Guard Service (Input validation) → RAG Pipeline (Embed → Search → Generate) →

Response (JSON to client)

MongoDB Collections

COLLECTION	MODEL	KEY FIELDS	PURPOSE
`chatSessions`	ChatSession	`sessionId` (unique), `messages[]`	Stores per-tab conversation threads with role/content/timestamp
`chatLogs`	ChatLog	`question`, `answer`, `status`, `timingMs`, `requestMeta`	Audit trail — every Q&A pair with full metadata for analytics
`adminUsers`	AdminUser	`email` (unique), `password` (bcrypt 12), `role`	Admin accounts for the dashboard
`adminSettings`	AdminSetting	`key` ("global"), `systemPrompt`, `chatbotEnabled`, `lastScrapeAt`	Singleton config controlling chatbot behavior

Chat Flow

End-to-end journey from user input to AI response

Session Initialization

When the ChatWidget mounts, it must establish a session before any messages can be sent.

1. **Check Visibility**
   `GET /api/chat/visibility` — returns `{ chatbotEnabled: boolean }`. If disabled, the widget stays hidden and no session is created.

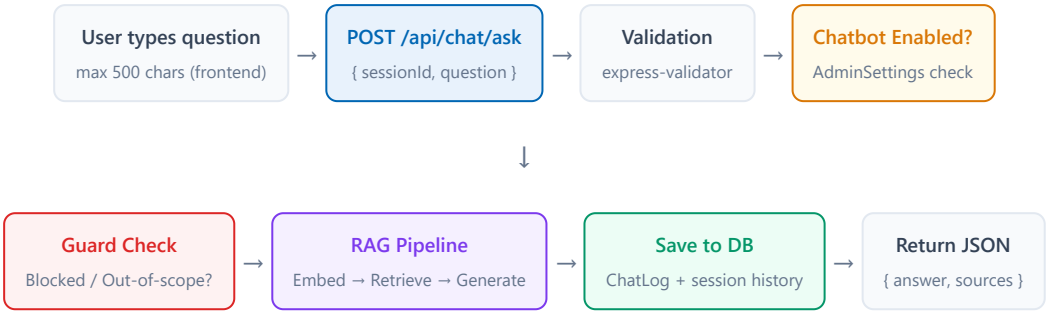
2. **Look for Existing Session**
   Checks `sessionStorage["chatWidgetSessionId"]`. If found, the stored UUID is reused (sessions survive page refreshes within the same tab).

3. **Create New Session (if needed)**
   `POST /api/chat/session` — server generates `crypto.randomUUID()`, upserts a `ChatSession` document in MongoDB, and returns the `sessionId`. A singleton promise prevents duplicate creation on concurrent renders.

> **Why sessionStorage?** Each browser tab gets its own independent chat session. Closing the tab naturally clears the session. This avoids stale conversations leaking across tabs.

Message Lifecycle

Every message follows this exact sequence through the backend:



Backend Controller Steps (chat.controller.js → askQuestion)

1. **Input Validation**
   `sessionId`: string, max 100 chars. `question`: string, 1–2000 chars. The frontend-side `.escape()` was intentionally removed to avoid mangling legitimate user input — the backend sanitizer handles XSS.

2. **Chatbot Enabled Check**
   Reads `chatbotEnabled` from `AdminSettings`. Returns HTTP 503 if disabled.

3. **Ensure Session**
   Upserts the session document — if the sessionId doesn't exist, it creates one. This handles edge cases where sessionStorage has a stale ID.

4. **Guard Check #1 — isBlockedQuery()**
   Tests against offensive, confidential, and prompt-injection patterns. If blocked, returns a safe fallback message and logs with `status: "fallback"`.

5. **Guard Check #2 — isOutOfScope()**
   Tests against out-of-scope patterns (code generation, recipes, sports, etc.). Returns a polite redirection message.

6. **Retrieve History**
   Loads conversation history from the ChatSession's `messages[]` array. This gives the AI context about the ongoing conversation.

7. **Run RAG**
   Calls `runRAG(question, history)` which embeds, retrieves, and generates the answer. See the RAG Pipeline section for details.

8. **Persist & Respond**
   Adds both user & assistant messages to the session. Creates a `ChatLog` with full metadata (timing, sources count, model info, IP, user-agent). Emits `analytics:updated` and `information:updated` via Socket.IO. Returns `{ answer, sources }`.

> **Session Recovery:** If the backend returns 404 ("session not found"), the frontend auto-retries once by clearing `sessionStorage` and creating a fresh session.

Quick Action Buttons

Three preset buttons appear when the conversation is empty. Clicking one sends it as if the user typed it:

"What jobs are available?" | "How do I apply?" | "What are the benefits?"

These are passed directly to `sendMessage(action)` — the same function used for typed messages. No special handling needed on the backend.

RAG Pipeline

Retrieval-Augmented Generation: Embed → Retrieve → Generate

End-to-End RAG Flow

User Question "What internships are available?" → **Embed** text-embedding-3-small → **Retrieve** Elasticsearch kNN → **Generate** GPT-4o-mini → **Answer** JSON response

Step 1 — Embed the Question

The user's question is converted into a 1536-dimensional vector using Azure OpenAI's `text-embedding-3-small` model via `getEmbeddingClient().embeddings.create()`.

```
// embedding.service.js
const response = await getEmbeddingClient().embeddings.create({
  model: AZURE_EMBEDDING_DEPLOYMENT,
  input: text
});
return response.data[0].embedding; // Float[1536]
```

This vector captures the semantic meaning of the question and will be compared against all stored content vectors.

Step 2 — Retrieve Relevant Chunks

The embedding vector is sent to Elasticsearch's kNN (k-Nearest Neighbors) search, which finds the most semantically similar content chunks.

```
// retrieval.service.js → searchRelevantChunks()
const result = await esClient.search({
  index: "unido_careers_index",
  knn: {
    field: "embedding",
    query_vector: embedding,
    k: TOP_K,              // default 5
    num_candidates: 50     // broader initial scan
  }
});
// Filter by MIN_RELEVANCE_SCORE (default 0.75)
```

PARAMETER	DEFAULT	ENV VARIABLE	PURPOSE
`TOP_K`	5	`RAG_TOP_K`	Maximum chunks returned
`MIN_RELEVANCE_SCORE`	0.75	`RAG_MIN_SCORE`	Cosine similarity threshold — rejects low-confidence matches
`num_candidates`	50	—	Elasticsearch evaluates this many candidates before picking top K

> **No relevant results?** If no chunks pass the minimum score threshold, the system returns "*I do not have enough information to answer that question*" without calling GPT — saving cost and preventing hallucination.

Step 3 — Generate the Answer

The retrieved chunks are formatted as context and sent alongside the question to GPT-4o-mini.

```
// generation.service.js
const messages = [
  { role: "system", content: systemPrompt },
  ...normalizedHistory,
  {
    role: "user",
    content: `QUESTION: ${question}\n\nCONTEXT:\n${contextText}`
  }
];

const response = await getChatClient().chat.completions.create({
  model: AZURE_OPENAI_DEPLOYMENT,  // gpt-4o-mini
  messages,
  temperature: 0.1  // near-deterministic
});
```

System Prompt (configurable via Admin UI)

The system prompt is loaded from `AdminSettings.systemPrompt`. Default:

```
"You are a UNIDO Careers Assistant. Answer ONLY from the provided CONTEXT.
If the context does not contain enough information, say you don't have
enough information. Do not make up answers."
```

Why temperature 0.1?

Near-zero temperature ensures the AI stays faithful to the retrieved context. For a careers information chatbot, factual accuracy is critical — creative or speculative answers could mislead job applicants.

Guard Service

Content filtering and prompt injection defense

Four-Layer Content Filter

Every user question passes through four pattern categories before reaching the RAG pipeline:

User Input → **Offensive** (Profanity, threats) → **Confidential** (PII, sensitive data) → **Injection** (Prompt manipulation) → **Out-of-Scope** (Unrelated topics) → ✓ RAG Pipeline

CATEGORY	ACTION	SAMPLE PATTERNS
OFFENSIVE	Blocked — returns safe fallback	Profanity, hate speech, threats, violence keywords, self-harm
CONFIDENTIAL	Blocked — returns safe fallback	"staff salary", "passport", "ssn", "credit card", "medical record", "employee data"
PROMPT INJECTION	Blocked — returns safe fallback	"ignore previous instructions", "you are now", "system:", "DAN jailbreak", "[INST]", "<<SYS>>", "forget everything"
OUT OF SCOPE	Polite redirection message	"write me code", "recipe", "weather", "sports", "stock market", "crypto", "homework"

Additional Checks

- Empty strings → blocked
- Strings over 2000 characters → blocked
- Each blocked query is logged with `status: "fallback"` and a `guardReason` for audit purposes

Contextual Responses

`buildBlockedAnswer(reason)` returns different user-friendly messages per reason type — an offensive query gets a different response than a prompt injection attempt.

Data Pipeline

Scraping, cleaning, chunking, embedding, and indexing

Pipeline Overview (pipeline.js)

The `runScraper()` function executes the full pipeline sequentially. This runs daily via cron or on-demand from the admin panel.

Scrape (Puppeteer + Cheerio) → **Clean HTML** (cleanerHTML.js) → **Chunk** (chunker.js) → **Embed** (OpenAI 1536D vectors) → **Index** (Elasticsearch bulk) → **Backup** (JSON files)

Phase 1 — Scraping

Six scrapers extract all content from the UNIDO Careers website using Puppeteer (headless Chrome):

SCRAPER	SOURCE	WHAT IT EXTRACTS
Homepage	`careers.unido.org/`	Hero title & description, CTA link, career category links
FAQs	`unido.org/employment/faqs`	Numbered Q&A pairs from FAQ page
Job Search	`careers.unido.org/search/`	Job title, location, department, grade, deadline, detail link. Paginates 25/page
Job Details	Individual job URLs	Full job listing: requisition ID, grade, duty station, all sections (responsibilities, qualifications, etc.)
Categories	Category page URLs	Category title, description, total jobs, job listings within each category
Category Content	Learn-more page URLs	Detailed paragraphs, lists, tables (grade/experience requirements) with 1.5s delay between pages

Phase 2 — HTML Cleaning

`cleanerHTML.js` uses **Cheerio** to load raw HTML and strip noise:

- Removes `<script>`, `<style>`, `<header>`, `<footer>`, `<nav>` elements
- Returns a Cheerio `$` instance for further DOM querying by the scrapers

Phase 3 — Chunking

The `chunker.js` module splits cleaned content into semantically meaningful chunks using sentence-based splitting with configurable overlap.

PARAMETER	DEFAULT	PURPOSE
`maxLength`	1000 chars	Maximum characters per chunk
`overlapSentences`	1	Sentences carried from previous chunk for context continuity

Chunk Types Produced

TYPE	ID PATTERN	DOCUMENT TYPE
Homepage	`homepage_hero_N`, `homepage_cta_1`	hero / cta / category
FAQs	`faq_{hash}_N`	faq
Job Summaries	`job_summary_{hash}_N`	job_summary
Job Details	`job_detail_{hash}_{section}_N`	job_detail (max 1200 chars)
Job Categories	`category_{hash}_N`	category_overview / category_job
Category Content	`category_content_{hash}_{section}_N`	category_content

Each chunk includes metadata: `chunk_index`, `total_chunks`, `content_length`, `embedding_version: "v1"`, `source: "unido_careers"`.

Phase 4 — Embedding & Indexing

1. **Ensure Index**
   Creates the `unido_careers_index` in Elasticsearch with `dense_vector` mapping (1536 dimensions, cosine similarity) if it doesn't exist.

2. **Clear Existing Data**
   `deleteByQuery({ match_all: {} })` removes all documents from the index to ensure a clean re-index.

3. **Generate Embeddings**
   For each chunk, `withEmbeddings()` calls Azure OpenAI to generate a 1536-dimensional vector. Processed one-by-one to respect rate limits.

4. **Bulk Index**
   All chunks (with embeddings) are indexed using Elasticsearch's bulk API for efficient writes.

5. **Save Backup**
   JSON files saved to local `data/` and `chunks/` folders with date-prefixed filenames (e.g., `180226_unido_faqs.json`).

> **Cron Schedule:** When `ENABLE_CRON=true`, the pipeline runs daily at midnight UTC via `node-cron` schedule `"0 0 * * *"`. Timezone is configurable via `CRON_TIMEZONE`.

Authentication Flow

Admin login, JWT verification, and route protection

Login Sequence

Admin Login Form (email + password) → POST /api/admin/auth/login (Rate limited: 5/min) → bcrypt.compare (12 rounds) → JWT Sign (HS256 · 25min expiry) → localStorage ("adminToken")

Backend Details (adminAuth.service.js)

1. **Find Active User**
   Queries `AdminUser.findOne({ email, isActive: true })`
2. **Constant-Time Defense**
   If user not found, hashes a dummy password anyway to prevent timing-based user enumeration attacks.
3. **Password Verification**
   `bcrypt.compare()` with 12 salt rounds (OWASP recommended).
4. **Token Generation**
   Signs JWT with `{ id, role: "admin", iat }`, algorithm HS256, expiry from `JWT_EXPIRES` (default 25m). Secret must be 32+ characters in production.

Token Verification Middleware

Every admin API request passes through `requireAdminAuth`:

1. **Extract Token**
   Reads `Authorization: Bearer <token>` header.
2. **Format Validation**
   Checks 3-part JWT format (header.payload.signature). Rejects malformed tokens before decoding.
3. **JWT Verify**
   `jwt.verify()` with `algorithms: ["HS256"]` and `maxAge: "30m"`.
4. **Role Check**
   Verifies `decoded.role === "admin"`. Returns 403 Forbidden with IP logged if role mismatch.

> **Security:** Expired tokens get a specific `TokenExpiredError` response (401). Other verification failures get a generic 401. This prevents information leakage about why a token was rejected.

Client-Side Route Protection (PrivateRoute.jsx)

A React wrapper component that guards admin routes:

1. Reads token from `localStorage.getItem("adminToken")`
2. Parses the JWT payload via base64 decode (no verification — server handles that)
3. Checks `exp` timestamp with a 30-second buffer for clock drift
4. Validates `role === "admin"`
5. If invalid → removes token → redirects to `/login`
6. If valid → renders the protected children

> **Defense in depth:** Client-side checks are for UX only. Every API call is independently verified by the server middleware. A tampered client-side token can't access real data.

Admin Panel Operations

Analytics, settings, scraping, exports, and conversation management

Analytics Dashboard

`GET /api/admin/analytics` computes metrics via MongoDB aggregation, optionally filtered by `fromDate` / `toDate`.

METRIC	SOURCE	COMPUTATION
Conversations	ChatSession	`countDocuments()` with date filter
Messages	ChatSession	Aggregate: unwind `messages[]`, group by role, sum
Unique Users	ChatLog	`distinct("requestMeta.ip")`
Avg Response (ms)	ChatLog	`$avg` of `timingMs.total`
Errors	ChatLog	`countDocuments({ status: "error" })`

Visualized with three Recharts pie charts: Total Data, Message Distribution, Errors vs Success.

Settings Management

Admin edits prompt (max 500 chars) → PUT /api/admin/settings { systemPrompt, chatbotEnabled } → MongoDB Update AdminSettings.key="global" → Socket.IO Emit chatbot:visibilityChanged

The `chatbotEnabled` toggle controls whether the ChatWidget is visible to end users. Changes propagate in **real-time** via Socket.IO — no page refresh required.

Scraping Control

1. **Admin clicks "Run Full Scrape & Reindex"**
   `POST /api/admin/scrape/trigger` → calls `triggerScrape()`
2. **State Machine: idle → running**
   If already running, returns `{ started: false }`. Otherwise, fires the pipeline asynchronously.
3. **Frontend Polls Status**
   `GET /api/admin/scrape/status` polled every 5 seconds while status is "running". Returns `{ lastStatus, lastError }`.
4. **Pipeline Completes**
   State transitions to `success` or `error`. Updates `lastScrapeAt` in AdminSettings.

CSV Export

Select date range & type (From / To / All or Conversations) → GET /api/admin/reports/chat-logs (Query params) → Server generates CSV (Content-Type: text/csv) → Browser download (Blob → <a> click)

CSV Columns

`createdAt`, `sessionId`, `status`, `question`, `answer`, `sourceCount`, `responseMs`, `ip`

The "conversations" type filters for `status: { $in: ["success", "fallback"] }`, excluding error-only logs. Limit: up to 10,000 rows.

All Information Table

`GET /api/admin/allInformation` returns paginated, searchable chat logs:

- **Pagination:** `page` (1–10000), `limit` (1–100, default 25)
- **Search:** Regex-based across `sessionId`, `question`, `answer`, `requestMeta.ip`, `status`, `createdAt` (formatted as dd/mm/yyyy)
- **ReDoS Protection:** Search input is escaped via `escapeRegExp()`
- **Sort:** `createdAt: -1` (newest first)
- **Real-time:** Socket.IO `information:updated` event triggers auto-refresh

Real-time Communication

Socket.IO events for live updates

Socket.IO Architecture

Socket.IO runs on the same Express server. Clients connect via WebSocket with polling fallback. Admin clients include their JWT in the `auth` handshake for elevated access.

Connection Lifecycle

1. **Client Connects**
   Singleton pattern via `getSocket()`. Transports: `["websocket", "polling"]`. Admin clients send `auth.token`.
2. **Server Auth Middleware**
   If `auth.token` is present, verifies JWT → sets `socket.isAdmin = true`. Invalid/missing token allows connection but marks `isAdmin = false`.
3. **Ready Event**
   Server emits `socket:ready` with timestamp on successful connection.
4. **Idle Disconnect**
   30-minute timeout — idle connections are automatically closed to free resources.

Event Reference

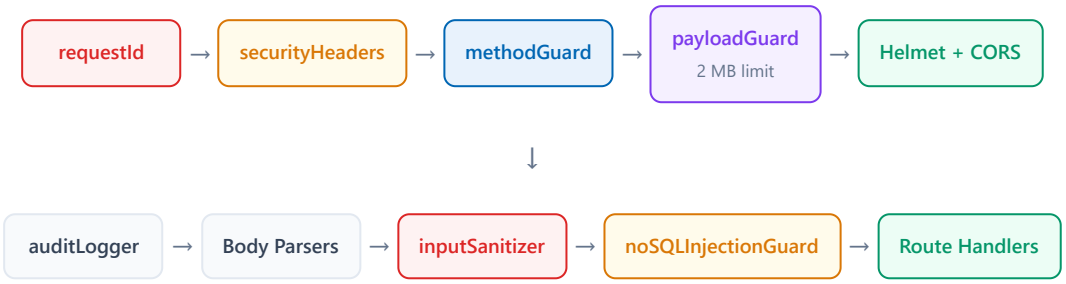
EVENT	DIRECTION	TRIGGERED BY	PURPOSE
`socket:ready`	Server → Client	Connection	Confirms connection with timestamp
`analytics:updated`	Server → All	Session create, chat log save, settings update	Triggers analytics dashboard refresh
`information:updated`	Server → All	Chat log created	Triggers All Information table refresh
`chatbot:visibilityChanged`	Server → All	Admin toggles chatbot on/off	ChatWidget shows/hides in real-time. Payload: `{ chatbotEnabled }`

Security Layer

Defense-in-depth middleware stack

Middleware Execution Order

Every request passes through this stack in order (server.js):



MIDDLEWARE	PURPOSE	ON FAILURE
requestId	Assigns `{timestamp}-{counter}` to every request for tracing	—
securityHeaders	X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Permissions-Policy (disables camera/mic/geo), Cache-Control: no-store	—
methodGuard	Allows only GET, POST, PUT, DELETE, OPTIONS, HEAD	405 Method Not Allowed
payloadGuard	Rejects Content-Length > 2 MB	413 Payload Too Large
auditLogger	Logs requestId, method, path, status, duration, IP, user-agent. Warns on errors (≥400) and slow requests (>5s)	—
inputSanitizer	Checks body & query for dangerous patterns: `<script>`, `javascript:`, `on*=`, `${}`, `{{}}`	400 Bad Request
noSQLInjectionGuard	Rejects body/query objects with keys starting with `$` (MongoDB operators like $gt, $ne)	400 Bad Request
globalErrorHandler	Catches unhandled errors. Generic message in production, detailed in development	500

Rate Limiting

LIMITER	ROUTE	WINDOW	MAX REQUESTS	NOTES
authLimiter	/api/admin/auth	60 seconds	**5**	Does not skip successful requests — brute-force defense
chatLimiter	/api/chat	60 seconds	**100**	Standard headers (RateLimit-*)
adminLimiter	/api/admin	60 seconds	**30**	Standard headers
speedLimiter	/api (all)	60 seconds	After 50 → **500ms delay**	express-slow-down: progressive delay

Helmet & CORS Configuration

Content Security Policy (CSP)

```
default-src 'self'
script-src  'self'
style-src   'self' 'unsafe-inline'
img-src     'self' data: https:
connect-src 'self'
object-src  'none'
frame-src   'none'
base-uri    'self'
form-action 'self'
```

CORS

- **Production origins:** `['https://careers.unido.org']`
- **Development origins:** `['http://localhost:5000', 'http://localhost:5173', 'http://localhost:5174']`
- Production blocks requests with no `Origin` header (CSRF prevention)
- Allowed methods: GET, POST, PUT, DELETE · Allowed headers: Content-Type, Authorization · Max age: 600s

Other Headers

- HSTS: 1 year, includeSubDomains, preload
- Cross-Origin-Embedder/Opener/Resource-Policy: `same-origin`
- DNS prefetch: disabled
- `X-Powered-By`: removed
- HTTPS redirect in production (via `x-forwarded-proto`)

API Reference

Complete list of REST endpoints

Public Endpoints (No Auth)

METHOD	PATH	BODY / PARAMS	RESPONSE
GET	`/health`	—	Health status of OpenAI + Elasticsearch
GET	`/api/chat/visibility`	—	`{ chatbotEnabled: boolean }`
POST	`/api/chat/session`	—	`{ success, sessionId }`
POST	`/api/chat/ask`	`{ sessionId, question }`	`{ answer, sources }`
POST	`/api/admin/auth/login`	`{ email, password }`	`{ token, admin }`
GET	`/api/docs.json`	—	OpenAPI spec
GET	`/api/docs`	—	Swagger UI (dev only)

Protected Endpoints (JWT Required)

METHOD	PATH	PARAMS / BODY	RESPONSE
POST	`/api/admin/auth/logout`	—	Success confirmation
GET	`/api/admin/analytics`	?fromDate, ?toDate (ISO 8601)	Metrics object (conversations, messages, uniqueUsers, etc.)
GET	`/api/admin/allInformation`	?page, ?limit, ?search	`{ logs[], pagination }`
GET	`/api/admin/settings`	—	`{ systemPrompt, chatbotEnabled, lastScrapeAt }`
PUT	`/api/admin/settings`	`{ systemPrompt?, chatbotEnabled? }`	Updated settings
POST	`/api/admin/scrape/trigger`	—	`{ started: boolean }`
GET	`/api/admin/scrape/status`	—	`{ lastStatus, lastError }`
GET	`/api/admin/reports/chat-logs`	?startDate, ?endDate, ?type, ?limit	CSV file download

Environment Variables

All configuration values needed to run the system

Backend (.env)

VARIABLE	REQUIRED	DEFAULT	PURPOSE
`PORT`	No	5000	Server port
`NODE_ENV`	No	development	"production" enables strict security
`MONGO_URI`	**Yes**	—	MongoDB connection string
`ELASTIC_NODE`	No	http://localhost:9200	Elasticsearch endpoint
`ELASTIC_USERNAME`	No	—	Elasticsearch basic auth
`ELASTIC_PASSWORD`	No	—	Elasticsearch basic auth
`AZURE_OPENAI_ENDPOINT`	**Yes**	—	Azure OpenAI endpoint URL
`AZURE_OPENAI_KEY`	**Yes**	—	Azure OpenAI API key
`AZURE_OPENAI_DEPLOYMENT`	**Yes**	—	GPT-4o-mini deployment name
`AZURE_EMBEDDING_DEPLOYMENT`	**Yes**	—	text-embedding-3-small deployment
`AZURE_OPENAI_API_VERSION`	No	2024-10-21	Azure API version
`JWT_SECRET`	**Yes**	—	JWT signing secret (32+ chars in prod)
`JWT_EXPIRES`	No	25m	JWT token expiration
`ADMIN_EMAIL`	**Yes**	—	Default admin email (bootstrapping)
`ADMIN_PASSWORD`	**Yes**	—	Default admin password (12+ chars)
`ENABLE_CRON`	No	false	"true" enables daily scraping cron
`CRON_TIMEZONE`	No	UTC	Cron job timezone
`RAG_TOP_K`	No	5	Max Elasticsearch results
`RAG_MIN_SCORE`	No	0.75	Min cosine similarity threshold

Frontend (.env)

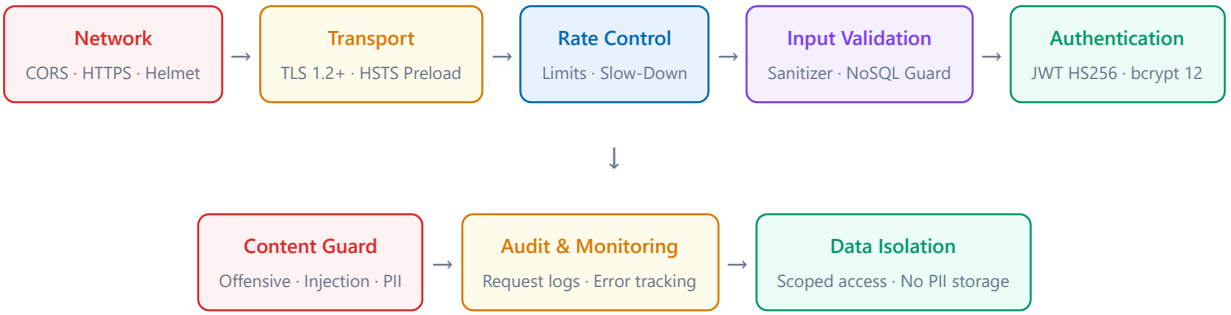
VARIABLE	DEFAULT	PURPOSE
`VITE_API_BASE_URL`	http://localhost:5000	Backend API base URL — used by all service functions

How Organizational Security Is Maintained

Multi-layer security posture protecting UNIDO data, users, and infrastructure

Security Architecture Overview

The chatbot employs a defense-in-depth strategy with seven distinct security layers, each designed to catch what the previous layer might miss:



1. Data Protection & Privacy

MEASURE	IMPLEMENTATION	WHY IT MATTERS
No PII Storage	The chatbot does not collect or store any personally identifiable information. Chat sessions are anonymous (UUID-based) and identified only by IP for rate limiting.	Ensures GDPR and UN data protection compliance — no user data to breach
PII Query Blocking	Guard service blocks questions containing confidential patterns: "staff salary", "passport", "ssn", "credit card", "medical record", "employee data"	Prevents users from attempting to extract sensitive organizational information via the AI
Session Isolation	Each browser tab gets a unique `sessionStorage`-based session. Sessions cannot cross-reference each other. Closing the tab destroys the session client-side.	Prevents conversation leakage between users or tabs
Response Grounding	AI answers ONLY from retrieved UNIDO content (RAG). Temperature 0.1 prevents hallucination. System prompt explicitly forbids made-up answers.	Eliminates risk of the AI revealing data it shouldn't know or generating false information
No External Data Leakage	CSP restricts `connect-src` to `'self'`. No third-party analytics, tracking pixels, or external API calls from the frontend.	User interactions never leave the UNIDO domain boundary

2. Authentication & Access Control

MEASURE	IMPLEMENTATION	STANDARD
Password Hashing	bcrypt with 12 salt rounds — each hash takes ~250ms, making brute-force computationally expensive	OWASP recommended minimum: 10 rounds
Timing Attack Defense	Login always hashes a dummy password even when user not found — constant response time regardless of user existence	OWASP Authentication Cheat Sheet
JWT Security	Algorithm locked to HS256 (prevents "none" algorithm attack). 25-minute expiry. 32+ character secret enforced in production. `maxAge: "30m"` server-side verification.	RFC 7519, OWASP JWT Cheat Sheet
Auth Rate Limiting	5 login attempts per minute per IP. Does NOT skip successful requests — prevents credential stuffing even with valid credentials.	OWASP Brute Force Prevention
Password Policy	Minimum 8 characters (API validation), 12+ characters recommended and warned on bootstrap. Max 128 characters to prevent bcrypt DoS.	NIST SP 800-63B
Role-Based Access	JWT contains `role: "admin"`. Middleware verifies role on every request. Non-admin tokens receive 403 with IP logged.	Principle of Least Privilege

3. Network & Transport Security

MEASURE	IMPLEMENTATION
HTTPS Enforcement	HSTS with 1-year max-age, includeSubDomains, preload. Automatic HTTP→HTTPS redirect in production via `x-forwarded-proto` check.
CORS Strict Origin	Production allows only `https://careers.unido.org`. Requests without `Origin` header blocked in production (prevents direct API abuse).
Frame Protection	`X-Frame-Options: DENY` and `frame-src: 'none'` — prevents clickjacking by forbidding embedding in iframes.
Content Sniffing	`X-Content-Type-Options: nosniff` — prevents browsers from MIME-sniffing responses away from declared content type.
Permissions Policy	Disables camera, microphone, and geolocation APIs — these are not needed and represent unnecessary attack surface.
Cross-Origin Isolation	`Cross-Origin-Embedder-Policy`, `Opener-Policy`, and `Resource-Policy` all set to `same-origin`.
Fingerprint Minimization	`X-Powered-By` header removed. DNS prefetch disabled. `Cache-Control: no-store` prevents sensitive data caching.

4. Input Security & Injection Prevention

ATTACK VECTOR	DEFENSE	LAYER
Cross-Site Scripting (XSS)	Input sanitizer blocks `<script>`, `javascript:`, `on*=` patterns. CSP blocks inline scripts. React's default escaping provides additional defense.	Middleware + CSP + Framework
NoSQL Injection	Dedicated `noSQLInjectionGuard` rejects any request body or query with keys starting with `$` (MongoDB operators like $gt, $ne, $or).	Middleware
Template Injection	Input sanitizer blocks `${}` and `{{}}` patterns — prevents server-side and client-side template injection.	Middleware
ReDoS (Regex DoS)	Search input in the admin table is escaped via `escapeRegExp()` before being used in MongoDB regex queries.	Service layer
Prompt Injection	Guard service blocks: "ignore previous instructions", "you are now", "system:", "DAN jailbreak", "[INST]", "<<SYS>>", "forget everything", and 10+ other patterns.	Application layer
Payload Overflow	`payloadGuard` rejects bodies > 2 MB. Input validation enforces max lengths on all fields (question: 2000, sessionId: 100, systemPrompt: 500, search: 200).	Middleware + Validation
HTTP Method Abuse	`methodGuard` allows only GET, POST, PUT, DELETE, OPTIONS, HEAD — blocks TRACE, CONNECT, PATCH, etc.	Middleware

5. Monitoring & Audit Trail

CAPABILITY	IMPLEMENTATION
Request Tracing	Every request gets a unique `requestId` (`{timestamp}-{counter}`). All logs reference this ID for end-to-end tracing.
Audit Logging	`auditLogger` middleware records: request ID, HTTP method, path, status code, response time, IP address, and user-agent for every request.
Error Alerting	Responses with status ≥ 400 are logged at WARN level. Slow requests (> 5 seconds) are flagged separately.
Chat Audit	Every Q&A interaction is stored in `ChatLog` with: question, answer, status, guard reason, response timing, source count, model used, IP, and user-agent.
Admin Activity	All admin API calls go through JWT middleware — the decoded token provides admin identity for every action. Failed auth attempts log IP addresses.
Security Events	Blocked queries (offensive, injection, PII) are logged with `status: "fallback"` and specific `guardReason` for security review.

6. AI-Specific Security

RISK	MITIGATION
Hallucination	RAG-only architecture with temperature 0.1. System prompt: "Answer ONLY from provided CONTEXT." If no relevant chunks found (score < 0.75), returns "I do not have enough information" without calling GPT.
Data Poisoning	Content is scraped only from official UNIDO domains (`careers.unido.org` , `unido.org`). No user-submitted content enters the knowledge base. Full re-index clears stale data.
Prompt Injection	Four-layer guard service filters input before it reaches the AI. Patterns block system prompt override attempts, DAN jailbreaks, and instruction manipulation.
Scope Creep	Out-of-scope pattern detection redirects users asking about code, recipes, sports, stocks, etc. The AI is scoped strictly to UNIDO careers content.
Token Abuse	Chat rate limit (100/min per IP) + input max length (2000 chars) + speed limiter (500ms delay after 50 requests) prevents excessive API/token usage.

7. OWASP Top 10 Coverage

OWASP CATEGORY	STATUS	CONTROLS
A01 — Broken Access Control	Mitigated	JWT + role check on every admin route. CORS strict origin. Frame protection.
A02 — Cryptographic Failures	Mitigated	bcrypt 12 rounds. HTTPS enforced. JWT HS256 with 32+ char secret. No PII stored.
A03 — Injection	Mitigated	Input sanitizer + NoSQL guard + express-validator + ReDoS-safe regex + prompt injection guard.
A04 — Insecure Design	Mitigated	Defense-in-depth: 7 middleware layers. Fail-safe defaults. Constant-time auth.
A05 — Security Misconfiguration	Mitigated	Helmet hardened. X-Powered-By removed. Production env enforces strict configs. assertOpenAIConfig() validates at startup.
A06 — Vulnerable Components	Ongoing	Modern dependencies (Express 5, React 19). Regular `npm audit` recommended.
A07 — Auth Failures	Mitigated	Rate limiting (5/min auth). Timing-safe comparison. Token expiry + maxAge. Failed attempts logged.
A08 — Data Integrity Failures	Mitigated	Content sourced only from official UNIDO domains. JWT signature verification. No deserialization of untrusted data.
A09 — Logging & Monitoring	Mitigated	auditLogger on every request. Error/slow request warnings. Chat audit trail. Security event logging.
A10 — SSRF	Mitigated	No user-supplied URLs are fetched. Scraping targets are hardcoded to UNIDO domains only.

Go-Live Requirements from UNIDO

Services and access credentials needed from UNIDO to deploy the chatbot to production

The following five services are **required** from UNIDO before the chatbot can go live. Each service plays a specific role in the system architecture — without any one of them the chatbot cannot function correctly in production.

Azure OpenAI API

Critical

What it is: Microsoft's enterprise-grade AI service that hosts large language models (LLMs) such as GPT-4o-mini and embedding models like text-embedding-3-small.
Why we need it: Azure OpenAI API powers the chatbot's core intelligence. It is responsible for two key capabilities:

- **Understanding questions** — The embedding model converts every user question into a 1536-dimensional vector that captures its semantic meaning, enabling the system to find the most relevant career content even when the user's phrasing doesn't match the exact words on the website.
- **Generating responses** — The GPT-4o-mini chat model reads the retrieved career content and composes accurate, human-like answers grounded strictly in UNIDO's official information. Temperature is set to 0.1 to minimize hallucination.

What UNIDO needs to provide:

CREDENTIAL / DETAIL	ENVIRONMENT VARIABLE
Azure OpenAI endpoint URL	`AZURE_OPENAI_ENDPOINT`
API key	`AZURE_OPENAI_API_KEY`
Chat model deployment name (GPT-4o-mini)	`AZURE_OPENAI_DEPLOYMENT`
Embedding model deployment name (text-embedding-3-small)	`AZURE_EMBEDDING_DEPLOYMENT`
API version (default: `2024-10-21`)	`AZURE_OPENAI_API_VERSION`

🔍 Elasticsearch (Vector Database)

Critical

What it is: A distributed search and analytics engine that supports k-nearest-neighbor (kNN) vector search, acting as the project's vector database.
Why we need it: Elasticsearch enables fast semantic search — the backbone of the RAG (Retrieval-Augmented Generation) pipeline. When a user asks a question:

1. The question is converted into a vector embedding via Azure OpenAI.
2. Elasticsearch performs a kNN search across all stored content vectors to find the **most semantically relevant** chunks of career information.
3. The top-matching chunks are passed to GPT-4o-mini as context so it can generate a grounded, factual answer.

Without Elasticsearch, the chatbot would have no way to locate relevant information from the hundreds of scraped pages and would be unable to answer any question.
What UNIDO needs to provide:

CREDENTIAL / DETAIL	ENVIRONMENT VARIABLE
Elasticsearch node URL (e.g., `https://es-cluster.unido.org:9200`)	`ELASTICSEARCH_NODE`
Username	`ELASTICSEARCH_USERNAME`
Password	`ELASTICSEARCH_PASSWORD`
Index name (default: `unido-careers`)	`ELASTICSEARCH_INDEX`

Note: Requires Elasticsearch 8.x+ with `dense_vector` field type and kNN search support (1536 dimensions). Options include Elastic Cloud, Azure-hosted Elasticsearch, or a self-managed cluster.

Azure Blob Storage

Critical

What it is: Microsoft Azure's scalable object storage service for unstructured data such as JSON files, HTML snapshots, and backups.

Why we need it: Azure Blob Storage serves as the persistent data layer between the scraping pipeline and the indexing pipeline:

- **Raw scraped content** — After the scraper collects job listings, FAQs, category pages, and homepage content from the UNIDO careers website, the raw JSON data is uploaded to Blob Storage. This creates a reliable, versioned archive of all scraped data.
- **Chunked content** — The chunker processes raw data into smaller, semantically meaningful pieces (chunks). These chunks are also stored in Blob Storage before being embedded and indexed into Elasticsearch.
- **Backup & re-indexing** — If Elasticsearch needs to be rebuilt (e.g., after a schema change or data corruption), the system can re-read all chunks from Blob Storage and re-index without having to re-scrape the entire website.

What UNIDO needs to provide:

CREDENTIAL / DETAIL	ENVIRONMENT VARIABLE
Azure Storage connection string	`AZURE_STORAGE_CONNECTION_STRING`
Container name (default: `unido-chatbot`)	`AZURE_STORAGE_CONTAINER`

🛡 Azure AI Content Safety / Guardrails

Important

What it is: Azure AI Content Safety is a cloud service that detects harmful or inappropriate content across text and images, providing severity scores across categories like hate, violence, sexual content, and self-harm.
Why we need it: While the chatbot already includes a multi-layer guard service (offensive patterns, prompt injection detection, PII blocking, and out-of-scope filtering), Azure AI Content Safety adds an **enterprise-grade, continuously-updated** safety layer:

- **Response filtering** — Before any AI-generated response is sent to the user, it can be scanned by Content Safety to ensure it does not contain inappropriate, harmful, or non-compliant content — even if the underlying model produces something unexpected.
- **Input screening** — User questions can be pre-screened to detect abusive or manipulative inputs that might bypass regex-based pattern matching.
- **Compliance assurance** — Provides auditable safety scores and categories for every interaction, helping UNIDO demonstrate responsible AI usage in line with UN digital governance standards.
- **Adaptive protection** — Unlike static regex patterns, Azure AI Content Safety uses ML models that are continuously updated by Microsoft to catch emerging threats and adversarial techniques.

What UNIDO needs to provide:

CREDENTIAL / DETAIL	ENVIRONMENT VARIABLE
Azure Content Safety endpoint URL	`AZURE_CONTENT_SAFETY_ENDPOINT`
API key	`AZURE_CONTENT_SAFETY_KEY`

Note: This service complements (not replaces) the existing built-in guard service. The two work in tandem — the local guard catches known patterns instantly, while Azure Content Safety catches novel or nuanced threats.

MongoDB (Credentials)

Critical

What it is: A NoSQL document database that stores structured, schema-flexible JSON-like documents. Used as the primary operational database for the chatbot.
Why we need it: MongoDB stores all the structured application data that powers the chatbot beyond just answering questions:

DATA TYPE	WHAT IT STORES	WHY IT MATTERS
Chat Sessions	Anonymous session records (UUID, IP, start time, message count)	Tracks active conversations and enables session continuity within a browser tab
Chat Logs	Every Q&A interaction: question, answer, status, guard reason, response time, source count, model used, IP, user-agent	Provides the analytics data shown in the admin dashboard, enables quality monitoring and security auditing
Admin Users	Admin accounts with bcrypt-hashed passwords and roles	Secures the admin portal with proper authentication
Admin Settings	System prompt, guard toggle states, configurable parameters	Allows the admin to customize chatbot behavior without code changes
Scraped Content Metadata	Job summaries, job details, FAQs, homepage content, category data with timestamps	Powers the "All Information" admin view and provides the raw data for chunking and indexing

What UNIDO needs to provide:

CREDENTIAL / DETAIL	ENVIRONMENT VARIABLE
MongoDB connection string (e.g., `mongodb+srv://user:pass@cluster.mongodb.net/unido`)	`MONGO_URI`

Note: Requires MongoDB 6+. Options include Azure Cosmos DB (MongoDB API), MongoDB Atlas, or a self-managed instance. A data retention policy should also be decided (e.g., purge chat logs older than 90 days).

Summary of Required Services

#	SERVICE	ROLE IN SYSTEM	PRIORITY
1	**Azure OpenAI API**	Understands questions (embeddings) and generates human-like responses (GPT-4o-mini)	Critical
2	**Elasticsearch**	Semantic vector search to find the most relevant career content from scraped data	Critical
3	**Azure Blob Storage**	Stores raw scraped content and chunks for processing, backup, and re-indexing	Critical
4	**Azure AI Content Safety**	ML-powered guardrails ensuring responses are safe, compliant, and free of harmful content	Important
5	**MongoDB**	Stores scraped content, chat logs, admin users, settings, and analytics data	Critical